query可靠性机制

目录

- 一、整体架构概述
  - 1.1 设计哲学总览：作为状态机的 Query
  - 1.2 双层入口函数
  - 1.3 循环状态 (State) 的设计意图
- 二、上下文压缩与溢出恢复
  - 2.1 主动压缩链路（Proactive → Autocompact）
  - 2.2 响应式压缩链路（Reactive）
  - 2.3 阻塞限制检查
- 三、API 重试与模型容错
  - 3.0 两层错误处理的边界在哪里
  - 3.1 `withRetry` 包装器（`services/api/withRetry.ts`）
  - 3.2 模型降级（Model Fallback）
- 四、Streaming 容错
  - 4.1 Streaming Tool Executor
  - 4.2 孤儿消息处理（Tombstoning）
  - 4.3 Tool Result Budget（工具结果大小限制）
- 五、Max Output Tokens 恢复
  - 5.1 级别一：Token 上限升级
  - 5.2 级别二：多轮恢复（`MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3`）
  - 5.3 级别三：暴露错误
- 六、Token 预算机制
  - 6.1 递减回报检测的设计意图
  - 6.2 `lastGlobalTurnTokens` 与轮次间累积的追踪
- 七、Stop Hooks 机制
  - 7.1 执行流程
  - 7.2 阻断处理
  - 7.3 Prevent Continuation
  - 7.4 Teammate 钩子链
- 八、用户中断处理
  - 8.1 流式中断（用户 Ctrl+C）
  - 8.2 工具执行中断
  - 8.3 Submit Interrupt（提交式中断）
- 九、异步预取与并行化
  - 9.1 Memory Prefetch
  - 9.2 Skill Discovery Prefetch
  - 9.3 Tool Use Summary
- 十、Tool Use 上下文刷新
- 十一、错误分类与处理

- 11.1 错误类型映射
- 11.2 错误暴露的安全保障
- 十二、特性开关汇总
  - 12.1 `querySource` 作为有类型的安全策略上下文
- 十三、设计哲学总结
  - 13.0 错误处理的"三层漏斗"哲学
  - 13.1 防御式流式处理
  - 13.2 暂挂（Withheld）模式
  - 13.3 不可变状态转移
  - 13.4 多层降级策略
  - 13.5 依赖注入：可测试性的刻意设计
  - 13.6 529 过载的"放大攻击"防护
  - 13.7 后台任务的"防雪崩"隔离
  - 13.8 QueryCheckpoint 的双层性能追踪体系
  - 13.9 工具执行中的"隐式依赖"感知
  - 13.10 Analytics 作为架构的第一等公民
  - 13.11 `using` 声明与资源管理的确定性
  - 13.12 死亡螺旋防护
- 十四、关键数据流图
- 十五、相关文件速查

一、整体架构概述

`query.ts` 是 Claude Code 的**主查询引擎**，负责驱动用户与 LLM 之间的完整对话循环。它以 `async generator` 的形式实现，支持流式输出（streaming），并在每次 API 调用之间插入大量可靠性机制，以应对上下文溢出、速率限制、模型过载、用户中断等运行时故障。

1.1 设计哲学总览：作为状态机的 Query

理解 query.ts 的关键不在于记住每一条恢复路径，而在于理解它**为什么**被设计成一个无限循环的**状态机**，而非顺序执行的函数链。这一选择背后有三条核心原则：

原则一：LLM 对话天然是非确定性的。 一次 API 调用可能因上下文超限而失败（需要压缩）、因速率限制而等待（需要重试）、因模型过载而需要降级（需要切换模型）、因用户按了 Ctrl+C 而中断（需要清理）。这些故障不可能在调用前全部预知，必须在运行时逐个应对。将整个流程包裹在 `while(true)` 循环中，使得每一种故障都有一个"继续迭代、重试"的统一出口，而无需为每种错误写独立的重试分支。

原则二：流式输出要求实时干预。 Claude Code 不是"等模型完整响应后再处理"的批处理系统，而是**在流式输出的过程中就开始执行工具**。这带来了一个根本的张力：一旦开始 yield 流式数据，控制流就已经分散到了调用方（SDK/REPL）。`async generator` 的设计解决了这个问题——`yield` 可以在流式输出的同时将控制权交回 query 循环，让循环继续处理下一个工具结果，而调用方继续消费已 yield 的数据。这是一种"协作式并发"（cooperative concurrency）：generator 和调用方各自推进，通过 `yield` 协调节奏。

原则三：不可变状态转移防止"幽灵 bug"。 query 循环中有 9 个 `continue` 站点，每个站点都需要**更新全部**跨迭代状态。如果使用可变字段赋值（如 `state.field = newValue`），某个 continue 站点忘记重置某个字段就会导致状态泄露，引发难以追踪的幽灵 bug。将**整个 `State` 对象替换为新对象**，使得每个 continue 站点要么显式设置每个

字段（通过 spread），要么继承旧值（通过 `...(previous)`）——遗漏的字段等于遗漏了一行代码，在代码审查中一目了然。

核心文件结构：

```
query.ts                          # 主查询循环（入口 + 核心 while(true) 循环）
├── query/stopHooks.ts            # Stop Hooks 执行器（回合结束时的钩子链）
├── query/tokenBudget.ts          # Token 预算跟踪与提前停止判断
├── services/api/withRetry.ts     # API 调用重试包装器
└── [其他支撑模块]
    autoCompact / reactiveCompact / contextCollapse / microcompact / snipCompact
    StreamingToolExecutor / runTools
```

1.2 双层入口函数

query.ts 故意将公共 API (`query`) 和内部循环体 (`queryLoop`) 分离为两个函数：

```
// 公开入口：负责清理 consumedCommandUuids
export async function* query(params: QueryParams): AsyncGenerator<...> {
  const consumedCommandUuids: string[] = []
  const terminal = yield* queryLoop(params, consumedCommandUuids)
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed')
  }
  return terminal
}

// 内部循环体：整个可靠性机制的核心载体
async function* queryLoop(params, consumedCommandUuids): AsyncGenerator<...> {
  // while(true) 无限循环，通过 continue 跳转继续迭代
  // 通过 return { reason: 'xxx' } 退出循环
}
```

`query()` 层的职责是"会话生命周期管理"：当 `queryLoop` 正常返回时（而非 throw 或 return），通知所有消耗的命令已完成。它是整个 generator 链的"收尾钩子"。`queryLoop()` 则承载所有运行时可靠性逻辑——这是两者分离的核心原因。如果将两者合并，可靠性逻辑和会话收尾逻辑就会纠缠在一起，增加测试和修改的复杂度。

关于 `yield*` 的使用：这里使用 `yield*` （委托生成）而非直接 `yield` ，是因为当 `queryLoop` 的内部 generator `throw` 时，异常会直接穿透 `query()` 传播到调用方，无需显式 try/catch。这提供了与普通函数相同的异常透明传播语义，使得 SDK 消费者（如 cowork/desktop）可以用统一的 `try/catch` 捕获所有异常。

PS：

- 如果 queryLoop 里抛出了异常，这个异常直接出现在调用方的 catch 里——调用方根本不知道 query 这个函数存在过
- 这和 return 传递返回值是同一个逻辑：yield* 让异常也走"直接穿透"这条路，而不是经过中间人转发

1.3 循环状态 (State) 的设计意图

整个可靠性机制建立在**跨迭代状态持久化**之上。`State` 类型将所有需要跨越循环迭代保存的 mutable 状态聚合在一起：

```
type State = {
  messages: Message[]                    // 对话历史
  toolUseContext: ToolUseContext         // 工具上下文
  autoCompactTracking: AutoCompactTrackingState | undefined  // 自动压缩状态
  maxOutputTokensRecoveryCount: number  // max_output_tokens 恢复计数
  hasAttemptedReactiveCompact: boolean  // 是否已尝试过响应式压缩
  maxOutputTokensOverride: number | undefined  // 输出 token 上限覆盖
  pendingToolUseSummary: Promise<...> | undefined  // 工具使用摘要（异步）
  stopHookActive: boolean | undefined   // Stop Hook 是否处于激活状态
  turnCount: number                     // 当前轮次计数
  transition: Continue | undefined      // 上一次迭代继续的原因（调试/测试用）
}
```

每次 `continue` 都通过 `state = { ...nextState }` **替换整个状态对象**，这是一种**不可变状态转移**模式，避免了散乱的局部变量赋值。

二、上下文压缩与溢出恢复

PS：详见“上下文管理”相关文档

这是 query.ts 中最复杂的可靠性子系统，分为**主动压缩（Proactive）** 和**响应式压缩（Reactive）** 两条路径。

2.1 主动压缩链路（Proactive → Autocompact）

在 API 调用之前，按顺序执行以下压缩步骤：

2.1.1 History Snip (`HISTORY_SNIP` 特性开关)

```
// restored-src/src/services/compact/snipCompact.ts
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages
  snipTokensFreed = snipResult.tokensFreed  // 回传给 autocompact 的阈值检查
}
```

- 目标：将"足够旧"且"不再被引用"的历史消息从上下文窗口中移除
- 关键：释放的 token 数量（ `snipTokensFreed` ）会被传递给 autocompact，使 autocompact 的阈值检查反映 snip 已移除的内容

PS：即截断功能，这是个很暴力的上下文操作方法，在当前版本的CC中**该功能是完全静默关闭**的

2.1.2 Microcompact（微压缩）

```
const microcompactResult = await deps.microcompact(messagesForQuery, toolUseContext,
querySource)
messagesForQuery = microcompactResult.messages
```

- **Cached Microcompact** (`CACHED_MICROCOMPACT` 特性)：对编辑类工具结果进行缓存优化，利用 API 返回的 `cache_deleted_input_tokens` 字段精确计算删除量
- 延迟到 API 响应后才 yield 边界消息，以便使用 API 实际报告的 `cache_deleted_input_tokens`

PS：即 cache_edit 能力，详见“上下文管理”相关文档

2.1.3 Context Collapse（上下文折叠，`CONTEXT_COLLAPSE` 特性）

```
if (feature('CONTEXT_COLLAPSE') && contextCollapse) {
  const collapseResult = await contextCollapse.applyCollapsesIfNeeded(...)
  messagesForQuery = collapseResult.messages
}
```

- 基于"提交日志"的折叠机制：折叠结果持久化在折叠存储中，而非直接修改 REPL 数组
- 这使得折叠在多轮之间持久化：每次 `projectView()` 在新的入口上重放提交日志

2.1.4 Autocompact（自动压缩）

```
const { compactionResult, consecutiveFailures } = await deps.autocompact(...)
```

- 当检测到 token 数量超过阈值时，对早期消息进行摘要压缩
- `consecutiveFailures` 字段记录连续失败次数，用于**熔断器（circuit breaker）** 逻辑
- 压缩成功后重置 `tracking` 状态：

```
tracking = {
  compacted: true,
  turnId: deps.uuid(),
  turnCounter: 0,          // 回合计数器重置
  consecutiveFailures: 0, // 连续失败计数重置
}
```

Analytics 埋点：压缩成功后记录详细的 token 统计（压缩前后 token 数、cache hit/miss 等），用于成本分析。

task_budget 跨压缩边界追踪：作为资源消耗的上限护栏，防止 Agent 在上下文压缩导致记忆丢失后，无意或有意地绕过派生数量限制，确保整个会话生命周期内（跨越任意次压缩）子 Agent 总数始终不超过配置的安全阈值

2.2 响应式压缩链路（Reactive）

当主动压缩未能防止 **413 Prompt Too Long** 错误时，触发响应式路径：

2.2.1 Withheld 机制（错误暂挂）

在流式循环中，可恢复的错误会被**暂挂（withheld）**，不立即向调用方暴露：

```
let withheld = false
if (contextCollapse?.isWithheldPromptTooLong(message, ...)) withheld = true
if (reactiveCompact?.isWithheldPromptTooLong(message)) withheld = true
if (mediaRecoveryEnabled && reactiveCompact?.isWithheldMediaSizeError(message)) withheld =
true
if (isWithheldMaxOutputTokens(message)) withheld = true
if (!withheld) { yield yieldMessage }  // 仅在非暂挂时 yield
```

> **设计意图**：若过早暴露错误，SDK 消费者（如 cowork/desktop）会在收到 `error` 字段时终止会话，导致恢复逻辑无人认领。

2.2.2 两阶段恢复

阶段一：Context Collapse 队列排空（仅当上一轮不是 `collapse_drain_retry` 时触发）

```
if (feature('CONTEXT_COLLAPSE') && state.transition?.reason !== 'collapse_drain_retry') {
  const drained = contextCollapse.recoverFromOverflow(messagesForQuery, querySource)
  if (drained.committed > 0) {
    state = { ...state, transition: { reason: 'collapse_drain_retry', committed:
drained.committed } }
    continue  // 重试 API 调用
  }
}
```

PS：核心思想是 不压缩整体对话，而是**将已标记为"可折叠"的消息块（比如已经被模型引用过、不再需要完整保留的中间结果）替换为更短的占位符**

阶段二：Reactive Compact（若阶段一未能解决）

```
const compacted = await reactiveCompact.tryReactiveCompact({
  hasAttempted: hasAttemptedReactiveCompact,
  querySource,
  messages: messagesForQuery,
  ...
})
if (compacted) {
  state = { ...state, transition: { reason: 'reactive_compact_retry' } }
  continue
}
```

PS：核心思想就是如果 context collapse 没能解决（没有可折叠单位，或折叠后仍然超限），就做一个全量摘要：将之前的整个对话历史压缩成一段短摘要

死锁防护：若两次恢复均失败，**不**落入 stop hooks——因为模型从未产生有效响应，stop hooks 会造成死亡螺旋：error → hook blocking → retry → error → …

2.3 阻塞限制检查

在**主动压缩关闭**时，进入 API 调用前会检查是否达到硬性阻塞限制：

```
if (!compactionResult && querySource !== 'compact' && querySource !== 'session_memory'
    && !(reactiveCompact?.isReactiveCompactEnabled() && isAutoCompactEnabled())
    && !collapseOwnsIt) {
  const { isAtBlockingLimit } = calculateTokenWarningState(
    tokenCountWithEstimation(messagesForQuery) - snipTokensFreed,
    toolUseContext.options.mainLoopModel,
  )
  if (isAtBlockingLimit) {
    yield createAssistantAPIErrorMessage({ content: PROMPT_TOO_LONG_ERROR_MESSAGE, error:
'invalid_request' })
    return { reason: 'blocking_limit' }
  }
}
```

- 跳过条件：压缩刚完成（已验证在阈值以下）、是 compact/session_memory 查询、启用了 reactive compact 或 context collapse

---

三、API 重试与模型容错

3.0 两层错误处理的边界在哪里

query.ts 中实际上存在两个不同的重试层，**需要重点关注划分边界**：

第一层（`withRetry.ts`）：处理**传输层和网关层**的错误——429 速率限制、529 过载、401/403 认证、408 超时、ECONNRESET/EPIPE 连接问题。这些错误的共同点是：它们与**请求本身的内容无关**，纯粹是基础设施层面的问题。重试相同的内容（完全相同的 messages）通常就能成功。
第二层（query.ts 循环）：处理**语义层**的错误——413 PTL、max_output_tokens、模型降级。这些错误的共同点是：它们与**请求的内容直接相关**，必须通过修改请求内容（压缩上下文、切换模型、注入恢复消息）才能解决。

这个分层的设计在于：**语义层的修改必须放弃当前请求（不能简单重试）**，而传输层的错误**可以重试相同请求**

3.1 `withRetry` 包装器（`services/api/withRetry.ts`）

```
async function* withRetry<T>(
  getClient: () => Promise<Anthropic>,
  operation: (client, attempt, context) => Promise<T>,
  options: RetryOptions,
): AsyncGenerator<SystemAPIErrorMessage, T>
```

3.1.1 重试策略

错误类型	处理策略
408 Request Timeout	重试（指数退避 + jitter）
409 Lock Timeout	重试
429 Rate Limit	重试（仅限非订阅用户；企业用户因 PAYG 可重试）
529 Overloaded	重试（`MAX_529_RETRIES=3`，触发 Model Fallback）
5xx Server Error	重试
APIConnectionError (ECONNRESET/EPIPE)	重试并禁用 keep-alive
401/403 认证错误	清除缓存 + 刷新 token 后重试
max_tokens 超出 context 限制	动态调低 `max_tokens` 后重试

3.1.2 退避公式

PS：指数退避 + 抖动（Exponential Backoff with Jitter）

```
const baseDelay = Math.min(BASE_DELAY_MS * 2^(attempt-1), maxDelayMs)  // maxDelayMs=32000
const jitter = Math.random() * 0.25 * baseDelay
return baseDelay + jitter
```

jitter 是 0.25 而非 0.5（Full Jitter）？ 这意味着**平均等待时间比理论值更长**，而非更短
这可能是一个有意为之的设计：API Gateway 的限流通常基于滑动窗口，平均等待时间偏长可以减少窗口内再次触发限流的概率。全抖动在低并发时表现更好，但在这个场景中（已有大量并发请求的客户端），稍微保守一些的退避策略可能更安全。

3.1.2.1 `RETRY_AFTER` 头部的优先级

值得注意的是，退避策略**优先使用服务器返回的** `Retry-After` 头部（如果有）：

```
if (retryAfterHeader) {
  return parseInt(retryAfterHeader, 10) * 1000  // 直接使用服务器建议
}
```

PS：服务器比客户端更清楚自身当前的负载状况

3.1.3 持久化重试模式（`UNATTENDED_RETRY`）

适用于长时间运行的无人值守会话（如 background agent / CI pipeline）：

```
const PERSISTENT_MAX_BACKOFF_MS = 5 * 60 * 1000  // 5 分钟上限
const PERSISTENT_RESET_CAP_MS = 6 * 60 * 60 * 1000  // 6 小时上限
const HEARTBEAT_INTERVAL_MS = 30_000  // 每 30s yield 一次防空闲信号
```

- 长睡眠被切分成 30 秒的小块，每块后检查 `AbortSignal`
- 期间定期 yield `{type:'system', subtype:'api_retry'}` 防止 host 将会话标记为空闲

PS：这类任务的特点是**宁可等 API 恢复也不期望出现失败**

3.1.4 Fast Mode 降级

Fast Mode 遇到 429/529 时的处理：

```
if (retryAfterMs < SHORT_RETRY_THRESHOLD_MS) {  // < 20s
  await sleep(retryAfterMs); continue  // 保持 fast mode，重试（短时间宁愿等待，降级后 cache 就失效了，损失很大）
} else {
  triggerFastModeCooldown(Date.now() + cooldownMs)  // 降级到标准速度
  retryContext.fastMode = false; continue
}
```

PS：

- Fast Mode 是 Claude Code 的一个推理加速特性，核心原理是通过 API 的 **Prompt Caching（上下文缓存复用）** 来减少每次请求的 token 消耗，从而降低延迟和成本
- 429 = 你被限速了等一等；529 = 服务器快塌了别添乱

3.1.5 前台/后台区分

仅以下 `querySource` 在 529 时重试（用户在前台等待）：

```
const FOREGROUND_529_RETRY_SOURCES = new Set([
  'repl_main_thread', 'sdk', 'agent:*', 'compact', 'hook_agent',
  'hook_prompt', 'verification_agent', 'side_question', 'auto_mode', ...
])
```

其他后台来源（如摘要生成、标题生成）遇到 529 **立即失败**，避免在容量过载时放大网关压力。

PS：

前台来源（用户有感，可重试），只有这 4 类：

1. REPL 主循环 — 用户在终端里打字，等着看输出
2. Agent 执行 — 包括自定义 Agent、内置 Agent、hook 里调用的 Agent

3. Compact — 上下文压缩，用户感知不到但任务本身是同步完成的
4. 同步安全判断 — auto_mode 分类器、side_question 提问、verification_agent
   本质就一条：有真实的人坐在屏幕前等着这个结果。

后台来源（用户无感，不可重试）：

- 标题/名称生成：会话标题、rename 新名字、Teleport 标题
- 记忆系统：记忆提取、相关性搜索、跨会话搜索、session_memory
- Agent 生命周期：创建 Agent、Agent 摘要、Agent 恢复
- Hook：prompt_suggestion、speculation、skill_improvement
- 外部集成：Chrome MCP、Web 搜索、web_fetch、mcp_datetime_parse
- 辅助分析：insights、auto_dream、away_summary、magic_docs、feedback
- 工具相关：tool_use_summary_generation、bash_extract_prefix、permission_explainer、model_validation
  本质：这个请求干不干成，用户根本不知道，也不会在意。失败了静悄悄吞掉就行，没必要在 API 过载时再去添乱。

3.2 模型降级（Model Fallback）

当连续 3 次 529 错误后，触发模型降级：

```
// withRetry.ts 中
if (consecutive529Errors >= MAX_529_RETRIES) {
  if (options.fallbackModel) {
    throw new FallbackTriggeredError(options.model, options.fallbackModel)
  }
}

// query.ts 中捕获
if (innerError instanceof FallbackTriggeredError && fallbackModel) {
  currentModel = fallbackModel
  attemptWithFallback = true  // 重试循环标志

  // 清除孤儿消息
  for (const msg of assistantMessages) {
    yield { type: 'tombstone', message: msg }  // 从 UI/Transcript 移除
  }

  // 丢弃旧 tool_use_id 的待处理工具结果
  streamingToolExecutor.discard()
  streamingToolExecutor = new StreamingToolExecutor(...)

  // Thinking signatures 是模型绑定的，降级模型无 capybara 签名权限
  if (process.env.USER_TYPE === 'ant') {
    messagesForQuery = stripSignatureBlocks(messagesForQuery)
  }

  yield createSystemMessage(`Switched to ${renderModelName(innerError.fallbackModel)} ...`,
'warning')
  continue
}
```

四、Streaming 容错

4.1 Streaming Tool Executor

当 `streamingToolExecution` 特性开启时，工具可以在模型响应流式输出的过程中**并行执行**，而非等待完整响应：

```
const streamingToolExecutor = new StreamingToolExecutor(
  toolUseContext.options.tools,
  canUseTool,
  toolUseContext,
)

// 在流式循环中实时注册工具
for (const toolBlock of msgToolUseBlocks) {
  streamingToolExecutor.addTool(toolBlock, message)
}

// 实时获取已完成工具的结果
for (const result of streamingToolExecutor.getCompletedResults()) {
  yield result.message
  toolResults.push(result.message)
}
```

PS：即模型流式输出多个并行工具调用时，**每当一个 tool_use 块完整输出，就立即开始执行该工具**，而非等待所有块都输出完毕后再统一执行

4.2 孤儿消息处理（Tombstoning）

当流式回退（streaming fallback）发生时，部分 assistant 消息（如 thinking blocks）具有无效签名，直接重试会导致 API 错误：

```
if (streamingFallbackOccured) {
  for (const msg of assistantMessages) {
    yield { type: 'tombstone', message: msg }  // 从 UI 中移除
  }
  logEvent('tengu_orphaned_messages_tombstoned', { orphanedMessageCount:
assistantMessages.length, ... })
  assistantMessages.length = 0
  streamingToolExecutor.discard()  // 丢弃旧 tool_use_id 的结果
  streamingToolExecutor = new StreamingToolExecutor(...)
}
```

PS：

- 流式回退（streaming fallback）：流式输出中途遇到错误时，丢弃所有已收到的半成品片段，直接用非流式模式重新发一次请求，最终只向用户呈现一条完整的 AssistantMessage
- 因为上面4.1 Streaming Tool Executor的原因，所以需要丢弃旧 tool_use_id 的结果，防止时序和执行混乱

4.3 Tool Result Budget（工具结果大小限制）

```
messagesForQuery = await applyToolResultBudget(
  messagesForQuery,
```

```
  toolUseContext.contentReplacementState,
  persistReplacements ? records => recordContentReplacement(records, toolUseContext.agentId)
: undefined,
  new Set(toolUseContext.options.tools.filter(t =>
!Number.isFinite(t.maxResultSizeChars)).map(t => t.name)),
)
```

- 对聚合工具结果大小执行**per-message预算**
- 与 microcompact 组合使用（两者互不干扰，因为 microcompact 仅按 `tool_use_id` 操作）
- 仅对 `agent:*` 和 `repl_main_thread` 查询源持久化替换记录（支持 `/resume` 和 AgentTool 恢复）

`applyToolResultBudget` 与 microcompact 的组合正交性：这两个压缩机制操作的是消息的不同维度——`applyToolResultBudget` 压缩的是**单个消息内**的工具结果大小（按字符数上限），microcompact 压缩的是**消息之间**的关系（按工具结果数量阈值和缓存状态）

仅对特定 querySource 持久化的原因：tool result 替换记录需要写入文件系统（`recordContentReplacement` 写 sidechain 或 session 文件）。对于临时性的 subagent 调用（如 `agent_summary`），这些记录会迅速变得无效（subagent 执行完毕后会话状态完全不同），写入磁盘是纯粹的浪费。通过限制持久化到"主会话"和"带 `/resume` 能力的 agent"，系统避免了不必要的磁盘 I/O。

五、Max Output Tokens 恢复

当模型输出达到 `max_output_tokens` 限制时的**三级恢复**：

5.1 级别一：Token 上限升级

仅当 Statsig 特性 `tengu_otk_slot_v1` 开启时，检测到使用了默认 8k 上限时，升级到 `ESCALATED_MAX_TOKENS`（64k）并重试（同一轮内）：

```
if (capEnabled && maxOutputTokensOverride === undefined
    && !process.env.CLAUDE_CODE_MAX_OUTPUT_TOKENS) {
  state = {
    ...state,
    maxOutputTokensOverride: ESCALATED_MAX_TOKENS,
    transition: { reason: 'max_output_tokens_escalate' },
  }
  continue
}
```

5.2 级别二：多轮恢复（`MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3`）

```
const recoveryMessage = createUserMessage({
  content: `Output token limit hit. Resume directly — no apology, no recap. ` +
           `Pick up mid-thought if that is where the cut happened. ` +
           `Break remaining work into smaller pieces.`,
  isMeta: true,
})
const next: State = {
  messages: [...messagesForQuery, ...assistantMessages, recoveryMessage],
  maxOutputTokensRecoveryCount: maxOutputTokensRecoveryCount + 1,
  ...
```

```
}
state = next; continue
```

PS：其实是**采用上下文动态插入/干涉的方式**

- **构造了一条指令进行提示**，塞进对话历史，然后 continue 重跑一遍 API：
  `你被截断了，别道歉，别回顾，直接从中断的地方继续。如果还没干完，把剩下的活拆成小块慢慢干`
- 这条消息是给模型，不是给用户看的（isMeta: true 意味着不出现在 UI 里）

5.3 级别三：暴露错误

若三个级别全部耗尽，向用户暴露被暂挂的错误消息：

```
if (isWithheldMaxOutputTokens(lastMessage)) {
  yield lastMessage  // 暴露错误
}
```

六、Token 预算机制

`query/tokenBudget.ts` 实现了一种"提前停止"机制，防止在 token 预算即将耗尽时继续消耗：

```
type BudgetTracker = {
  continuationCount: number   // 已自动继续的轮次
  lastDeltaTokens: number      // 与上一轮相比的 token 增量
  lastGlobalTurnTokens: number // 上轮总 token 数
  startedAt: number
}

function checkTokenBudget(tracker, agentId, budget, globalTurnTokens): TokenBudgetDecision
```

继续条件：

```
  /**
   * 递减判断（Diminishing Returns Detection）
   * 三个条件同时满足才触发：
   *   1. 已经连续检查过至少 3 次（本轮对话已经跑了好几轮）
   *   2. 这一轮产生的 token < 500（本轮输出很少）
   *   3. 上一轮产生的 token < 500（上一轮输出也很少）
   *
   * 含义：模型已经连续多轮只吐出一点点内容，说明任务快收尾了，
   * 继续让模型跑只会浪费 budget，应该停止这一轮。
   */
  const isDiminishing =
    tracker.continuationCount >= 3 &&      // ① 连续检查次数达标
    deltaSinceLastCheck < DIMINISHING_THRESHOLD &&  // ② 本轮增量 < 500 tokens
    tracker.lastDeltaTokens < DIMINISHING_THRESHOLD  // ③ 上一轮增量 < 500 tokens

  /**
   * 通过判断：尚未递减 且 未达到 budget 的 90%
   * → 允许模型继续工作，给用户一个 nudge 提示当前消耗进度。
   *
   * 反之任一条件满足就停止：
```

```
 *    - 已经递减（模型输出越来越短，快收尾了）
 *    - 已超过 90% budget（剩余空间不多了）
 */
if (!isDiminishing && turnTokens < budget * COMPLETION_THRESHOLD) {  // < 90%
  return { action: 'continue', nudgeMessage: '...' }
}
```

停止条件：

- Token 使用超过 90% 预算，或
- 连续 3 轮增量均 < 500 tokens（**递减回报检测**），或
- 无预算（`agentId` 子代理场景）

自动继续时的 meta 消息：向用户显示进度百分比：

```
Token budget continuation #2: 67% (335,000 / 500,000)
```

6.1 递减回报检测的设计意图

`checkTokenBudget` 中的"递减回报检测"（`continuationCount >= 3 && deltaSinceLastCheck < 500`）是一个**自适应停止机制**，它承认了一个现实：Agent 的工作产出不是线性随 token 消耗增长的。

具体来说：Agent 在前几个 chunk 中解决了大部分问题（"easy wins"），后续 chunk 主要在做"边缘情况处理"和"重复验证"。如果连续 3 轮的 token 增量都小于 500，说明模型每轮只添加了很少的新内容——这可能意味着：

1. 工作已接近完成（最好情况），或
2. 模型陷入了重复劳动或原地踏步（最坏情况）

代码选择了**保守停止**——宁可提前终止，也不让模型无限消耗预算。这是一种务实的设计哲学：在不可预测的 Agent 行为面前，预算护栏比精确的完成度判断更可靠。

PS：

- 比较有设计感的地方是**用 token 输出速率来判断模型是处于工作状态还是摸鱼状态**
- 但是这个设计比较暴力 -> 可以考虑用上下文动态注入的方法进行LLM行为的干涉

6.2 `lastGlobalTurnTokens` 与轮次间累积的追踪

`BudgetTracker.lastGlobalTurnTokens` 的设计揭示了一个问题：**token 预算的消耗是在哪里计算的?**

`getTurnOutputTokens()` 来自 `bootstrap/state.ts`，它记录的是**整个 turn 的总 token 消耗**（包括所有 API round）。但 `deltaSinceLastCheck = globalTurnTokens - tracker.lastGlobalTurnTokens` 是在计算**两个相邻检查点之间**的增量。这意味着：

- 如果一个 turn 包含多次 API 调用（多轮工具执行），所有这些调用的 token 都会累积到 `globalTurnTokens`
- 但 `lastDeltaTokens` 记录的只是"从上一个 budget 检查点到现在的增量"，而非"上一个 turn 的总消耗"

PS：主要是**瞬时变化检测**，即 `deltaSinceLastCheck` 关心的是"这一轮比上一轮多消耗了多少"，而不是"这一轮的总量是多少"。后者由 `budget * COMPLETION_THRESHOLD` 绝对阈值来处理。

七、Stop Hooks 机制

`query/stopHooks.ts` 在每次工具调用完成后（`needsFollowUp=false`）执行：

```
const stopHookResult = yield* handleStopHooks(
  messagesForQuery, assistantMessages, systemPrompt, userContext,
  systemContext, toolUseContext, querySource, stopHookActive,
)
```

stopHookResult 结构：

```
  type StopHookResult = {
    blockingErrors: Message[]      // hook 报告的错误消息（可累积多条）
    preventContinuation: boolean // hook 明确要求终止
  }
```

7.1 执行流程

1. **保存 CacheSafeParams**：供 `/btw` 和 SDK `control_request` 使用
2. **Template Job 分类**：每次回合后更新 `state.json`
3. **后台任务**（fire-and-forget）：
   - Prompt Suggestion
   - Memory Extraction（`EXTRACT_MEMORIES`）
   - Auto Dream
4. **Stop Hook 执行**：核心阻断逻辑

PS：`/btw` 和 SDK `control_request` 需要发起一个新 API 请求，但又要和主循环的 cache 复用——Stop Hooks 就在每轮结束时偷偷把主循环的 API 请求参数快照下来，供这两个场景直接拿来用，省掉重复创建 prompt cache 的开销

- `/btw`：主循环在干活的时候，用户不中断它，直接弹出一个 side question 获取答案
- `control_request`：SDK 宿主应用（如 IDE 插件、外部调度器）可以远程向正在运行的 Claude Code 会话发一个 control_request，效果和 `/btw` 完全一样：读主循环的上下文，发起一个单轮 fork 查询，返回答案

7.2 阻断处理

```
if (stopHookResult.blockingErrors.length > 0) {
  state = {
    messages: [...messagesForQuery, ...assistantMessages, ...stopHookResult.blockingErrors],
    stopHookActive: true,  // 防止无限递归
    transition: { reason: 'stop_hook_blocking' },
  }
  continue  // 将 blocking error 注入下一轮，模型看到后自行修正
}
```

无限循环防护：`hasAttemptedReactiveCompact` 保持不重置。若 reactive compact 已经失败过一次（表明上下文确实超限），stop hook 的 blocking error 再次触发 compact 会进入死循环——此防护将 `hasAttemptedReactiveCompact` 保持为 `true`，避免重复压缩尝试。

`hasAttemptedReactiveCompact` 持久化的深层含义：这个字段的语义不是"曾经尝试过"，而是"压缩对当前的上下文无效"。一旦它被设为 `true`，后续的 stop hook blocking error 就不能再触发新的压缩尝试——因为已经证明压缩无法解决问题，继续尝试只会浪费 API 调用并可能扩大上下文。

这实际上是在将 stop hook 的 blocking error 和 reactive compact 的失败进行关联记忆：**stop hook 认为可以通过修正模型行为来解决错误，但如果 reactive compact 已经表明问题不在模型行为而在上下文规模，stop hook 的**

干预就是无效的。

PS: REACTIVE_COMPACT 在当前版本中是默认关闭的，这还是测试阶段的功能

7.3 Prevent Continuation

若 hook 明确要求停止：

```
if (stopHookResult.preventContinuation) {
  return { reason: 'stop_hook_prevented' }
}
```

7.4 Teammate 钩子链

对于 team 模式中的 teammate，**在所有普通 Stop Hooks 全部通过（无 blocking error、无 preventContinuation）之后**，额外执行：

- `TaskCompleted` hooks（针对 teammate 拥有的 in_progress 任务）
  - 查任务列表，找出**这个 teammate 负责的、状态为进行中的任务**
  - 为每个任务执行一次 TaskCompleted hook
- `TeammateIdle` hooks
  - 所有 TaskCompleted hooks 跑完之后，执行一次 TeammateIdle hook。这个 hook 不关联任何具体任务，它关心的是"这个 teammate 整体是否空闲"

八、用户中断处理

8.1 流式中断（用户 Ctrl+C）

触发条件：`toolUseContext.abortController.signal.aborted === true`（用户按 ESC 拒绝工具 / Ctrl+C / SDK 发起 abort）
处理原则：给已开始的工具一个交代，不留孤儿 `tool_use` 块

```
/**
 * 第一步：消费已启动工具的剩余结果
 *
 * 背景：流式输出中，tool_use 块一旦完整输出就立即开始执行工具。
 * 用户中断时，部分工具可能：
 *   - 还在队列中等待（未开始）
 *   - 正在执行中（in-flight）
 *
 * getRemainingResults() 遍历所有已提交的工具：
 *   - 队列中未开始的：直接跳过（不执行）
 *   - 正在执行的：等待完成，然后生成合成 tool_result：
 *       "<tool_use_error>Error: Interrupted by user</tool_use_error>"
 *
 * 关键：如果不调用 getRemainingResults()，这些 tool_use 块就永远没有
 * 对应的 tool_result 块，导致消息树结构不完整，API 下一轮会困惑。
 */
if (toolUseContext.abortController.signal.aborted) {
  if (streamingToolExecutor) {
    // 主动拉取剩余结果（驱动 executor 完成 in-flight 工具并生成 synthetic error）
    for await (const update of streamingToolExecutor.getRemainingResults()) {
```

```
      if (update.message) yield update.message
    }
  } else {
    // 非流式工具执行模式（无 streamingToolExecutor）：
    // 找到所有 tool_use 块，逐个注入 "Interrupted by user" 的 tool_result
    yield* yieldMissingToolResultBlocks(
      assistantMessages,
      'Interrupted by user',
    )
  }

  /**
   * 第二步：Computer Use 清理（仅 CHICAGO_MCP 主线程）
   *
   * Computer Use 模式下：
   *   - 主线程持有一个全进程锁（isLockHeldLocally）
   *   - 主线程持有一个被遮罩的窗口（isHidden）
   *
   * 用户中断时需要：
   *   - 释放锁（让其他任务可以继续）
   *   - 取消遮罩（恢复窗口可见）
   *
   * 仅限主线程：subagent 的 cleanup 会错误地释放主线程的锁，
   * 导致主线程后续看到 isLockHeldLocally()===false，不再发送 exit 通知。
   */
  if (feature('CHICAGO_MCP') && !toolUseContext.agentId) {
    try {
      const { cleanupComputerUseAfterTurn } = await import(
        './utils/computerUse/cleanup.js'
      )
      await cleanupComputerUseAfterTurn(toolUseContext)
    } catch {
      // 静默失败：这是 dogfooding 清理，不在关键路径上
    }
  }

  /**
   * 第三步：发送用户中断消息
   *
   * 注意：submit-interrupt 场景（用户提交一个新消息同时中断当前轮）
   * 不需要这条消息——因为用户提交的下一条消息本身就已经提供了足够上下文，
   * 多发一条 "Interrupted by user" 会造成信息噪音。
   * 判断依据：abort reason === 'interrupt'（由 submitInterrupt 触发）
   */
  if (toolUseContext.abortController.signal.reason !== 'interrupt') {
    yield createUserInterruptionMessage({
      toolUse: false,  // 打断的是模型思考/输出，不是有 tool 的轮次
    })
  }

  return { reason: 'aborted_streaming' }
}
```

8.2 工具执行中断

```
if (toolUseContext.abortController.signal.aborted) {
  // ... 清理逻辑同 8.1
  if (toolUseContext.abortController.signal.reason !== 'interrupt') {
    yield createUserInterruptionMessage({ toolUse: true })
  }
  if (maxTurns && nextTurnCountOnAbort > maxTurns) {
    yield createAttachmentMessage({ type: 'max_turns_reached', ... })
  }
  return { reason: 'aborted_tools' }
}
```

8.3 Submit Interrupt（提交式中断）

跳过中断消息的发送（`reason !== 'interrupt'`），因为后续排队的用户消息已提供了足够的上下文。

九、异步预取与并行化

9.1 Memory Prefetch

```
using pendingMemoryPrefetch = startRelevantMemoryPrefetch(state.messages, toolUseContext)
// `using` 声明确保所有退出路径自动释放资源

// 在 API 流式循环中：消耗预取结果（仅在 settled 时）
if (pendingMemoryPrefetch.settledAt !== null && pendingMemoryPrefetch.consumedOnIteration
=== -1) {
  const memoryAttachments = filterDuplicateMemoryAttachments(
    await pendingMemoryPrefetch.promise,
    toolUseContext.readFileState,
  )
  for (const memAttachment of memoryAttachments) {
    yield createAttachmentMessage(memAttachment)
    toolResults.push(msg)
  }
  pendingMemoryPrefetch.consumedOnIteration = turnCount - 1
}
```

- 使用 `using`（TC39 Resource Management）确保 `pendingMemoryPrefetch` 在所有退出路径上被清理
  - 无论 query 循环以什么方式退出，后台内存搜索都会被取消，日志都会被记录
- `consumedOnIteration` 防止同一迭代周期内重复消费（yield 给用户）——注意这个字段在**迭代之间**比较（`turnCount - 1`），而非在同一迭代内比较

```
什么时候 prefetch 会被"消费"（yield 给用户）？

1. prefetch 的 Promise 已经有结果了（settledAt !== null）
2. 之前没有被消费过（consumedOnIteration === -1）

消费之后，立刻把当前轮次记录下来（consumedOnIteration = turnCount - 1）。

为什么要用 turnCount - 1？

因为 prefetch 只能在工具执行完毕之后才会被检查和消费。来看一轮 query 循环的内部顺序：

第 N 轮
├── 预检阶段（SNIP / Microcompact / Autocompact）
├── API 流式循环 ← prefetch 在这里 settle（Promise 有结果了）
├── 工具执行（StreamingToolExecutor / runTools）← 这里是唯一检查 prefetch 的地方
└── → continue → 进入第 N+1 轮

prefetch 在 API 流式循环中就 settle 了，但代码检查 prefetch
的位置在"工具执行之后"——工具执行本身发生在同一轮里，所以 prefetch 实际上是在第 N
轮的工具执行完毕处被消费的，不是等到第 N+1 轮。
```

- `readFileState` 过滤掉模型已经处理过的文件状态——这是跨迭代的累积过滤，确保即使 prefetch 结果在多次迭代后才被消费，也不会重复注入模型已经"知道"的文件状态。

9.2 Skill Discovery Prefetch

```
const pendingSkillPrefetch = skillPrefetch?.startSkillDiscoveryPrefetch(
  null, messages, toolUseContext,
)
// 在工具执行期间异步消耗
```

PS：

- 注意这一段没有具体的代码实现，是根据接口和注释来推断的，以下重要的技术细节并不知道：
  - startSkillDiscoveryPrefetch 内部具体怎么实现（用的什么模型、什么 prompt）
  - findWritePivot 的具体判断逻辑
  - collectSkillDiscoveryPrefetch 怎么计算 hidden_by_main_turn 的布尔值
  - 它搜索的 skill 来源是什么（文件系统？向量检索？）
- Skill Discovery Prefetch 是一个后台异步发现 + 随工具结果一起消费的机制：在可能涉及写操作的工具轮次中，通过 findWritePivot 判断触发后台 Skill Discovery，在模型推理和工具执行的同时悄悄准备好可能用到的 skill 附件，最后随工具结果一并注入，并用 hidden_by_main_turn 标记确保用户永远感知不到发现延迟——把原本每轮必付的阻塞成本，变成了一个几乎免费的副作用

9.3 Tool Use Summary

```
nextPendingToolUseSummary = generateToolUseSummary({
  tools: toolInfoForSummary,
  signal: toolUseContext.abortController.signal,
  isNonInteractiveSession: toolUseContext.options.isNonInteractiveSession,
  lastAssistantText,
})
.then(summary => summary ? createToolUseSummaryMessage(summary, toolUseIds) : null)
.catch(() => null)
```

每一轮工具执行完毕后，把"执行了哪些工具、干了什么"打包发给 Haiku（小模型），让 Haiku 生成一行人类可读的摘要（比如 "Fixed NPE in AuthService"），显示在移动端 UI 里：

- 结果在下一轮的 `pendingToolUseSummary` 处被 yield
- 子代理（`toolUseContext.agentId`）跳过此步骤（不在移动端 UI 显示）

PS：型看不到这条摘要（context不更改），是用户独享的信息（UI会显示）

十、Tool Use 上下文刷新

```
if (updatedToolUseContext.options.refreshTools) {
  const refreshedTools = updatedToolUseContext.options.refreshTools()
  if (refreshedTools !== updatedToolUseContext.options.tools) {
    updatedToolUseContext = {
      ...updatedToolUseContext,
      options: { ...updatedToolUseContext.options, tools: refreshedTools },
    }
  }
}
```

- 每次需要工具时调用这个函数，始终拿到最新状态
- 确保在多轮之间新连接的 MCP 服务器能及时可用。

为什么 `refreshTools` 是函数而非值

`toolUseContext.options.refreshTools` 是一个**函数**（`() => Tools[]`），而非直接存储工具数组的快照。这一设计选择背后的动机是：**工具列表在整个 turn 执行期间可能发生变化**。

典型场景：用户在工具执行期间新连接了一个 MCP server。MCP server 的连接是异步事件，可以在任意时刻发生。如果 `refreshTools` 是一个快照值而非函数，那么 query 循环需要在每个可能的退出路径上重新调用它，才能保证看到最新的工具列表——这容易遗漏且不优雅。

将其设计为函数后，query 循环只需在需要工具列表时**按需调用**（`refreshTools()`），每次调用返回当前最新状态——这本质上是**延迟求值（lazy evaluation）** 在工具管理中的应用：不在 turn 开始时确定工具列表，而是在每次需要时获取最新值。

十一、错误分类与处理

11.1 错误类型映射

错误	处理方式	退出原因
`ImageSizeError` / `ImageResizeError`	yield 友好消息	`image_error`
模型运行时异常	yield 孤儿 tool_result + 真实错误消息	`model_error`
`FallbackTriggeredError`	切换模型，重试	继续循环
`PROMPT_TOO_LONG` (pre-call)	yield 错误消息	`blocking_limit`
413 withheld → 恢复失败	暴露 withheld 错误	`prompt_too_long`
Media 错误恢复失败	暴露 withheld 错误	`image_error`
Stop hook blocking	注入错误消息，继续	继续循环
工具执行中断	生成合成 tool_result	`aborted_tools`
`hook_stopped_continuation`	停止	`hook_stopped`
max_turns 到达	yield 附件消息	`max_turns`
Stop hook prevented	停止	`stop_hook_prevented`

11.2 错误暴露的安全保障

```
// 防止暴露误导性的 "[Request interrupted by user]"
// SDK 消费者可能看到 phantom interrupt 而忽视真实原因
```

```
yield createAssistantAPIErrorMessage({ content: errorMessage })
logAntError('Query error', error)  // ants 模式下 loud logging
return { reason: 'model_error', error }
```

PS：

- 这是一个错误归因的透明性保障，**把"看起来像用户取消、实际上是 SDK bug"的 phantom interrupt 暴露为真实错误，防止用户误判**，从而帮助真正的 bug 被发现和修复
- 真实的情况中可能是 Node.js 运行时抛出异常（比如某个方法不存在），但 SDK 最外层没有精细处理，统一包装成了 "Request interrupted by user" -> queryModelWithStreaming 在极少数情况下会直接抛出异常（而非正常 yield 错误）

十二、特性开关汇总

特性名	作用	相关文件
`REACTIVE_COMPACT`	启用响应式压缩	`services/compact/reactiveCompact.js`
`CONTEXT_COLLAPSE`	启用上下文折叠	`services/contextCollapse/index.js`
`HISTORY_SNIP`	启用历史消息裁剪	`services/compact/snipCompact.js`
`CACHED_MICROCOMPACT`	启用缓存微压缩	`compact.ts`
`TOKEN_BUDGET`	启用 token 预算提前停止	`query/tokenBudget.ts`
`UNATTENDED_RETRY`	启用持久化重试	`services/api/withRetry.ts`
`BG_SESSIONS`	启用后台任务摘要	`utils/taskSummary.js`
`CHICAGO_MCP`	启用 MCP 清理钩子	`utils/computerUse/cleanup.js`
`EXTRACT_MEMORIES`	启用记忆提取	`services/extractMemories/`
`EXPERIMENTAL_SKILL_SEARCH`	启用技能预搜索	`services/skillSearch/prefetch.js`
`TEMPLATES`	启用模板任务分类	`jobs/classifier.js`

所有特性均通过 `feature('NAME')` 访问，由 `bun:bundle` 的 tree-shaking 机制确保未启用的特性字符串不进入外部构建。

12.1 `querySource` 作为有类型的安全策略上下文

`querySource` 是一个**命令式标签**（imperative label），它回答的是"谁在调用 query()"这个问题，而不仅仅是一个字符串标识符。它被用作策略分派的依据：

- `"compact"` / `"session_memory"`：子代理，跳过阻塞限制检查（死锁防护）
- `"repl_main_thread"` / `"sdk"`：主会话，保存 CacheSafeParams
- `"agent:*"`：子代理，持久化 tool result 替换记录
- 后台来源：529 立即失败，不重试

这种设计的关键是：**调用方比 query 循环本身更清楚自己的语义**。query 循环不需要理解"compact 代理是什么"，只需要知道"compact 来源的调用需要豁免阻塞检查"——这个策略知识是调用方传递给 query 的，而非 query 自己推导的。

十三、设计哲学总结

> 在阅读以下各节之前，请先回顾第一节的**"三大设计原则"：状态机模式、统一错误恢复出口、以及协作式并发**。这些原则贯穿整个 query.ts，是理解一切具体机制背后动机的基础。

13.0 错误处理的"三层漏斗"哲学

query.ts 对错误的处理遵循一个**统一的"三层漏斗"哲学**，而非为每种错误写独立分支：

- **第一层（Withhold，暂挂）**：在流式循环中捕获可恢复的错误，但不暴露给调用方。原因是 SDK 消费者可能在看到 `error` 字段时终止会话，使后续恢复逻辑永远无法执行。
- **第二层（Recover，恢复）**：在 turn 结束后尝试自动恢复：PTL 走 collapse drain → reactive compact → 最终暴露；max_output_tokens 走 8k→64k 升级 → 多轮恢复消息 → 最终暴露；529 走指数退避 → 模型降级 → 最终暴露。
- **第三层（Expose，暴露）**：所有恢复路径耗尽后，暴露错误并退出。此时调用 `executeStopFailureHooks`（而非完整的 stop hooks 链），防止模型从未产生有效响应时 stop hooks 注入更多 token 造成的死亡螺旋。

13.1 防御式流式处理

与传统的"先完整调用 API 再处理结果"模式不同，query.ts 在流式输出过程中**实时**处理工具、错误和中断。这种设计的优势在于：

- **并行化**：工具在模型仍在输出时开始执行
- **低延迟**：错误在产生时立即被捕获，而非等待完整响应
- **资源安全**：长时间运行的任务可被即时中断

13.2 暂挂（Withheld）模式

对于可恢复的错误（413 PTL、max_output_tokens、media size），选择**暂挂**而非立即暴露，原因是 SDK 消费者会在看到 `error` 字段时终止会话，使恢复逻辑永远无法执行。暂挂机制确保了恢复路径有足够的机会完成。

13.3 不可变状态转移

所有循环状态的更新都通过**替换整个 State 对象**完成：

```
state = { ...state, newField: newValue }
```

而非逐字段修改，这使得：

- 所有 continue 跳转点的状态结构一致
- 便于测试通过 `state.transition` 字段验证恢复路径
- 避免了"某些字段在某些分支未被正确重置"的 bug

13.4 多层降级策略

每种错误类型都设计了**至少两层**的降级路径：

- Prompt Too Long: `pre-call 阻止` → `context collapse 排空` → `reactive compact` → `暴露错误`
- Max Output Tokens: `8k→64k 升级` → `多轮恢复消息` → `暴露错误`
- 529 过载: `重试×3` → `模型降级` → `暴露错误`

13.5 依赖注入：可测试性的刻意设计

query.ts 的 `deps.ts` 实现了依赖注入（DI）模式，专门用于解决测试问题：当时有 6-8 个测试文件各自用 `spyOn` 拦截不同模块的函数，导致 import + spy 样板代码泛滥。

```
// deps.ts 的设计原则:
// 1. 类型从真实实现推导 (typeof fn), 确保签名永远同步
// 2. 生产环境用 productionDeps() 获取真实实现
```

```
// 3. 测试通过 QueryParams.deps 注入 fake 实现
// 4. 作用域故意收窄（仅 4 个依赖），证明模式可行后再扩展
```

这是一种"最小可行 DI"：不在整个系统中引入 IOC 容器，而是在最需要可测试性的模块（query.ts）中提供一个薄薄的注入层。`deps` 对象本身是一个普通 TypeScript 对象（而非 class/容器），使得注入无需任何框架支持。

13.6 529 过载的"放大攻击"防护

529 错误的处理逻辑中隐的全局设计考量：当 API Gateway 处于容量过载（529）状态时，**每一次重试请求实际上加重了网关的负担**，形成正反馈循环——这与普通的 429 速率限制不同，后者是本地配额耗尽，重试不会影响他人。

代码中 `FOREGROUND_529_RETRY_SOURCES` 的设计正是对这一问题的直接应对：只有用户在前台等待结果的查询来源才会重试 529；所有后台任务（摘要生成、标题生成、分类器等）遇到 529 **立即失败**，绝不重试。这将 529 时的请求放大倍数降到最低——因为后台任务失败不会影响用户体验，而前台用户请求可以短暂等待后重试。

同时注意，在 529 触发模型降级（fallback）时，只有**非订阅用户**（`!isClaudeAISubscriber()`）或非自定义 Opus 模型才降级。付费用户不走降级路径——他们的企业协议通常包含 PAYG（用后付费），重试比降级更合理。

13.7 后台任务的"防雪崩"隔离

与 529 过载防护相关的另一个设计点：后台任务（`session_memory` 查询、`compact` 代理等）在遭遇 context overflow 时会触发死锁——因为这些子进程本身需要运行才能**减少** token 使用量，如果在被调用时就因为 token 过多而阻止，就会永远无法执行。

代码中多处体现了这一隔离原则：

- 阻塞限制检查跳过了 `compact` 和 `session_memory` 查询来源
- microcompact 跳过了 `compact` 和 `session_memory` 查询来源（防止子代理和父代理同时尝试压缩互相等待）

这本质上是一种**资源顺序死锁预防**：如果 A 需要 B 先完成才能运行，而 B 也需要 A 先完成才能运行，系统就会卡死。通过让"消耗资源的子任务"免于被资源限制阻止，避免了这一经典死锁模式。

13.8 QueryCheckpoint 的双层性能追踪体系

query.ts 中散布的 `queryCheckpoint()` 调用实际上属于一个**双层性能追踪体系**，面向不同的使用场景：

第一层：Headless Profiler（`headlessProfilerCheckpoint`）追踪的是 **headless 模式下，用户从敲完回车到看到第一个字** 的全链路延迟分解。这类数据被发送到一个集中的监控系统，供 oncall 工程师判断是否需要扩容或回滚。

第二层：Query Profiler（`queryCheckpoint` + `utils/queryProfiler.ts`）追踪**单次 query 内部各阶段耗时**，定位"这次慢在哪"。这类数据通过 `CLAUDE_CODE_PROFILE_QUERY=1` 环境变量开启，在本地开发时输出到调试日志。

两者的分层设计避免了"用一个工具同时满足监控和调试两个需求，结果两边都做不好"的问题。监控需要低 overhead、聚合性好、不干扰生产服务；调试需要细粒度、即时反馈、可本地分析。

13.9 工具执行中的"隐式依赖"感知

StreamingToolExecutor 中 Bash 错误触发级联取消（sibling cancellation）的设计，揭示了代码对**隐式依赖**的深刻认知：

- 多个 Bash 命令之间通常存在隐式的文件/目录依赖关系，`hasErrored` 标志的级联取消机制正是对这一隐式依赖的编码：通过让第一个 Bash 错误"杀死"所有正在执行的兄弟工具，系统承认了 Bash 命令之间的依赖关系，并在第一时间止损。

- 但这并不意味着所有工具都遵循同一规则——读操作工具（Read/WebFetch/Grep/Glob）是**并发安全**的，它们之间不存在隐式依赖，可以并行运行。这是一种精细的并发控制策略：对不确定是否有依赖的操作保守（串行），对确定无依赖的操作激进（并行）。

13.10 Analytics 作为架构的第一等公民

整个 query.ts 中散布着大量 `logEvent()` 调用——不是在开发后期添加的"事后埋点"，而是系统架构的一部分

在功能开发的同一时间就设计好对应的埋点事件，而不是上线后才回头补。每一个关键的业务决策点（autocompact 触发、模型降级、token budget 停止、流式工具执行）都同步输出结构化的数据字段。这些字段从一开始就被设计来回答具体的产品问题（比如“压缩到底省了多少成本”、“token budget 是主动停还是被动停”），产品团队查数据就能做决策，而不是靠猜或看日志。产品逻辑和埋点逻辑是同一套代码、同一个 PR、同一个 Review 完成的——字段命名统一、数据格式一致、上线前就验证好、历史数据可追溯。这是把“数据驱动决策”这件事，从嘴上说说变成了代码里写死的制度。

13.11 `using` 声明与资源管理的确定性

Memory Prefetch 使用 TC39 `using` 声明（资源管理）而非手动清理，这是一种**确定性资源释放**的保证：

```
using pendingMemoryPrefetch = startRelevantMemoryPrefetch(state.messages, toolUseContext)
```

`using` 是 JavaScript 的确定性销毁机制（类似于 Rust 的 `Drop` 或 Python 的 `with`）。无论代码通过 `return`、`throw` 还是 `continue` 退出 query 循环，`pendingMemoryPrefetch` 的清理逻辑都会被调用。这消除了"资源泄漏"的一整类 bug——开发者无需记住在每个退出路径上手动调用清理函数。

在 query.ts 这样有 9 个不同退出路径的复杂循环中，确定性清理尤为重要。任何一条退出路径遗漏清理，都会在长会话中累积泄漏。

13.12 死亡螺旋防护

多处代码明确避免在**模型未产生有效响应**时落入 stop hooks：

```
// Do NOT fall through to stop hooks: the model never produced a valid response,
// so hooks have nothing meaningful to evaluate. Running stop hooks
// on prompt-too-long creates a death spiral:
// error → hook blocking → retry → error → …
yield lastMessage
void executeStopFailureHooks(lastMessage, toolUseContext)
return { reason: isWithheldMedia ? 'image_error' : 'prompt_too_long' }
```

类似地，`hasAttemptedReactiveCompact` 的持久化设计防止了 stop hook blocking error 触发的 compact 进入无限循环。

十四、关键数据流图

```
用户输入
  │
  ▼
┌──────────────────────────────────────────────────────────┐
│  queryLoop: while(true)                                  │
│  ┌──────────────────────────────────────────────────┐    │
│  │ 预检阶段:                                         │    │
```

```
│  │  1. Memory Prefetch (async, non-blocking)          │  │
│  │  2. Skill Discovery Prefetch (async)              │  │
│  │  3. HISTORY_SNIP ──→ 释放 token 预算               │  │
│  │  4. Microcompact ──→ 缓存优化                      │  │
│  │  5. Context Collapse ──→ 折叠                      │  │
│  │  6. Autocompact ──→ 摘要压缩                       │  │
│  │  7. [pre-call] 阻塞限制检查                        │  │
│  └──────────────────────────────────────────────────┘  │
│                          │
│                          ▼
│  ┌──────────────────────────────────────────────────┐  │
│  │ API 调用阶段（withRetry 包装）                      │  │
│  │  for await (message of deps.callModel())          │  │
│  │   ├─ 流式工具注册 → StreamingToolExecutor         │  │
│  │   ├─ 实时工具执行 → getCompletedResults()          │  │
│  │   ├─ Withheld 检查（PTL/媒体/MOT）                 │  │
│  │   └─ 孤儿消息 Tombstoning（fallback 时）           │  │
│  │                                                  │  │
│  │  错误处理:                                         │  │
│  │   ├─ 529 → 指数退避 → FallbackTriggeredError      │  │
│  │   ├─ 429/401/408/409/5xx → 指数退避重试            │  │
│  │   └─ max_tokens 溢出 → 动态调低 max_tokens 重试   │  │
│  └──────────────────────────────────────────────────┘  │
│                          │
│                          ▼
│  ┌──────────────────────────────────────────────────┐  │
│  │ 后处理阶段:                                        │  │
│  │   ├─ 用户中断检查 → 合成 tool_result               │  │
│  │   ├─ Post-sampling Hooks (fire-and-forget)        │  │
│  │   ├─ needsFollowUp=false?                         │  │
│  │   │   ├─ PTL 恢复: collapse drain → reactive compact │
│  │   │   ├─ MaxOutputTokens: 升级→多轮恢复→暴露      │  │
│  │   │   ├─ Stop Hooks → blockingErrors → continue   │  │
│  │   │   ├─ Token Budget → continue/stop             │  │
│  │   │   └─ return { reason: 'completed' }           │  │
│  │   └─ needsFollowUp=true?                          │  │
│  │       ├─ 工具执行（Streaming / Blocking）          │  │
│  │       ├─ Memory Prefetch 消费                      │  │
│  │       ├─ Skill Discovery 消费                      │  │
│  │       ├─ MCP 工具刷新                              │  │
│  │       └─ continue（下一轮）                        │  │
│  └──────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────┘
```

十五、相关文件速查

文件	职责
`restored-src/src/query.ts`	主循环、上下文管理、流式容错、恢复路径编排
`restored-src/src/query/stopHooks.ts`	Stop/TaskCompleted/TeammateIdle hooks 执行
`restored-src/src/query/tokenBudget.ts`	Token 预算跟踪与提前停止决策

文件	职责
`restored-src/src/query/deps.ts`	依赖注入（`deps.microcompact` / `deps.autocompact` / `deps.callModel`）
`restored-src/src/query/config.ts`	不可变配置快照（Statsig gates 除外）
`restored-src/src/services/api/withRetry.ts`	API 重试包装器、退避策略、模型降级
`restored-src/src/services/compact/reactiveCompact.ts`	响应式压缩（413/媒体错误恢复）
`restored-src/src/services/compact/autoCompact.ts`	主动压缩（轮次阈值触发）
`restored-src/src/services/compact/compact.ts`	`buildPostCompactMessages` 压缩结果构建
`restored-src/src/services/compact/snipCompact.ts`	历史消息裁剪
`restored-src/src/services/compact/microcompact.ts`	微压缩 / Cached Microcompact
`restored-src/src/services/contextCollapse/index.ts`	上下文折叠
`restored-src/src/services/tools/StreamingToolExecutor.ts`	流式工具并行执行器
`restored-src/src/services/tools/toolOrchestration.ts`	`runTools` 阻塞式工具执行
`restored-src/src/utils/hooks.js` / `postSamplingHooks.ts`	Hook 系统
`restored-src/src/utils/tokenBudget.js`	`getBudgetContinuationMessage` 进度消息
`restored-src/src/bootstrap/state.ts`	全局 token 预算状态